

CSC3003S Capstone Project — Stage One

Risk [30 Marks]

Project Abbrev	UAV-Path	Client/Supervisor + email	Patrick Marais <patrick@cs.uct.ac.za>
Date	08 August 2025	Tutor + email	Raaziq Parkar <prkraa002@myuct.ac.za>
Team Members	Kenneth Baloyi	Max Mkhabela	Lethabo Neo

Risk description [1]	Consequence [1]	Prob [½]	Impact [½]	Strategy (ARTA) [½]	Metric [½]+ monitoring [½]	Management [1½]
There is a steep learning curve with low-level 3D rendering libraries such as PyRender, OpenGL, or VTK. Integrating these libraries with optimization algorithms and ensuring real-time or interactive performance can introduce performance bottlenecks or integration bugs.	We may experience significant delays, reduced quality of the visualization module, or may need to switch to simpler libraries mid-project, compromising on interactivity or realism.	Medium to High.	High impact because visualization is a core deliverable for this project.	AVOIDANCE + TRANSFER We should start with higher-level libraries that are easy to use for initial development and if needed, we should transfer low-level rendering tasks to a teammate with prior experience or seek external mentorship from a tutor or supervisor.	Metric: Number of core visualization features implemented on schedule. Monitor this by using a feature checklist ticking off features that are implemented keeping track of ones that still need implementation.	We must schedule early prototyping sprints specifically for testing rendering tools. We must document all findings to support library choice. If complexity exceeds threshold, for example: if no interactive render working by week 4, then we would switch to a backup library. We must ensure that there is an ongoing visual output even if it isn't final.
We might attempt to include advanced features like dynamic obstacle avoidance, real-time sensor simulation, or multiple UAV coordination. This can go beyond the project's original scope and time constraints.	The deliverables may be incomplete, rushed, or fail to meet academic evaluation standards due to fragmented focus. We could become overwhelmed, thus leading to a burnout and poor software quality.	Medium	High	REDUCTION We must define a minimum viable product early, with fixed, achievable goals: One UAV at a time with a static environment and basic optimization. We should log all new feature ideas in a goal-list. We should reject or delay additions unless they serve the MVP	Metric: Number of features added outside original scope. Monitor this by keeping a feature-scope document shared with all team members. Whenever a new feature is proposed, log: - Who proposed it - Reason for inclusion - Whether it's accepted, delayed, or rejected	During our weekly meetings, we must review new ideas or feature requests against the MVP criteria. We can use a traffic light system for features: 1. Part of MVP, 2. Nice-to-have (later), 3. Out-of-scope.

Error in comparative metrics calculations (total path length, total vertical distance and estimated flight duration) and data visitations due to incorrect mathematical logic or flawed data handling.	The system would provide misleading analysis, failing its primary purpose. This would lead to users distrusting our systems and making all our features useless	Medium	High	REDUCTION As this is a critical component we will enforce Test driven Development approach for all analytical functions. Calculations will be validated against manually computed results	Metric: Code coverage percentage for modules containing calculations. Number of failed unit tests after new commit Use continuous integration (if available) to ensure regression errors are caught early.	We will create a comprehensive suite of unit tests for all mathematical and logical functions. Peer code reviews will be mandatory for any pull request that modifies calculation logic. This will ensure data correctness. Document all formulas and assumptions used in metric calculations to enable reproducibility and transparency.
Data incompatibility or corruption. For example, different Python versions may require different module versions or data structures. The system may also fail to load or may misinterpret trajectory or 3D model files (e.g., .json, .obj, .ply) if they are corrupted, malformed, or deviate from the expected format.	The system may crash, fail to load essential inputs (like flight paths or 3D models), or load them incorrectly — causing erratic UAV camera behaviour or incorrect visual outputs.	Medium	High	REDUCTION Implement robust error handling and input validation for all file parsing routines. Use “try-except” blocks, schema validation (e.g., for JSON), and format-specific sanity checks. Ensure modules used	Metric: % of malformed test files the system correctly identifies and rejects with user friendly error message. Number of app crashes logged that are linked to file loading Monitor this by using crash logs and validation test reports to monitor parser stability.	Develop a dedicated data validation module for file format used. Ensure the system provides clear error messages to guide users when files are invalid. Use virtual environments <code>venv</code> or <code>conda</code> to standardize Python/module versions. Document all file format specifications and supported versions for future developers or users.

				are compatible with the chosen Python version and clearly document environment dependencies.		
Low computing power available to handle real-time 3D rendering, especially on personal or older devices. The system may struggle with rendering UAV camera visuals smoothly.	Poor system performance. Rendering may take too long or cause some components of the system to be unusable while heavy task is running. Lagging during camera movements or image capturer	High	High	REDUCTION Use school Computers from the computer science department which have dedicated GPUs. Also apply some optimization techniques	Metric: rendering time per frame. Monitor this by tracking the rendering time per frame, can use built-in timers to track time taken per frame	Optimise the rendering pipeline to ensure efficiency, do multiple testing profiling the system regular to identify performance bottlenecks
Poor or inadequate testing	Software might have undiscover deadlocks leading to system crushing during runtime. Calculator for comparative metrics such as total path length or total vertical length might be faulty leading to incorrect result. System may fail to detect artefacts	Medium	High	REDUCTION Develop a well-defined test plan that is to be followed during the duration of the project, modularise code to allow for easy testing	Metric: % of features/scenarios tested or % tests passed Monitor this by keeping track of all the features that have been tested and all the test that have been passed	Identify the critical components and assign testing tasks per module with deadlines. Create a testing checklist to ensure all the critical components are tested. Allocate time buffer before each deliverable for testing.

Instructions

1. Complete the table with **realistic** risks to get marks.
2. You need at least 5 risks, you may have more.
3. Remove these instructions from your submission on Amathuba.

Follow-up

1. Bring this document to your next meeting with your client (or email it beforehand) and ask if it adequately accounts for possible issues.
2. Resolve and note the solutions to any issues.